



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

Laboratorio de:

Bases de Datos Distribuidas

Práctica No.:

01

Grupo No.:

5

Integrantes:

- Guillermo Cevallos
- Juan Chuga
- Karen Mosquera

Tema:

Planteamiento del diseño del proyecto

Objetivos:

- Describir un escenario real mediante una narrativa de negocio clara y estructurada.
- Diseñar el modelo entidad-relación que represente de forma gráfica y precisa la estructura lógica del escenario.
- Representar el modelo relacional mediante un grafo relacional que evidencie la integridad y consistencia de los datos.

Marco teórico:

El Modelo Entidad-Relación (MER) es una herramienta de diseño conceptual utilizada en bases de datos para representar de manera gráfica la estructura de la información de un sistema. Este modelo permite identificar las entidades, los atributos y las relaciones, las cuales muestran cómo se vinculan entre sí las entidades dentro del sistema.

Por otro lado, el Modelo Relacional o Grafo Relacional es la representación lógica derivada del MER, donde las entidades y relaciones se transforman en tablas. Además, se establecen claves primarias para identificar de manera única cada registro y claves



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

foráneas para relacionar unas tablas con otras. Este modelo sirve como base para la implementación e interpretación de la estructura en un Sistema Gestor de Bases de Datos.

Desarrollo de la práctica:

Planteamiento del escenario:

El siguiente escenario fue planteado por el modelo de gran tamaño Gemini 3 (Google, 2026):

“La empresa TechFix Solutions desea implementar una base de datos distribuida para coordinar sus operaciones de reparación y mantenimiento de hardware. El sistema debe permitir que cada centro de servicio gestione sus órdenes y personal de manera independiente.

1. Estructura de la Red de Servicios

La compañía opera mediante dos sedes principales identificadas con los códigos S01 (Norte) y S02 (Sur). De cada sede se requiere almacenar un código de sede, un nombre identificativo, la dirección física y un teléfono de contacto para emergencias.

2. Gestión de Personal Técnico

En cada sede trabaja un equipo de Técnicos asignados de forma exclusiva (un técnico no puede trabajar en dos sedes simultáneamente). Para cada técnico se registra: un código de empleado, cédula, nombre completo y su especialidad principal.

3. Registro de Clientes

Los Clientes son las personas que llevan sus dispositivos a las sedes para solicitar servicios de reparación o mantenimiento. De cada cliente se requiere almacenar su número de identificación, nombre completo, número de teléfono y correo electrónico. Estos datos solo serán usados como documentación, debido a que el Cliente no debería poder participar directamente con el sistema.

4. Órdenes de Reparación y Proceso Diferenciado

Los clientes llevan sus dispositivos a las sedes, donde se generan Órdenes de Servicio. Cada orden registra un número de folio, la fecha de ingreso, la descripción del fallo y el estado actual.



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

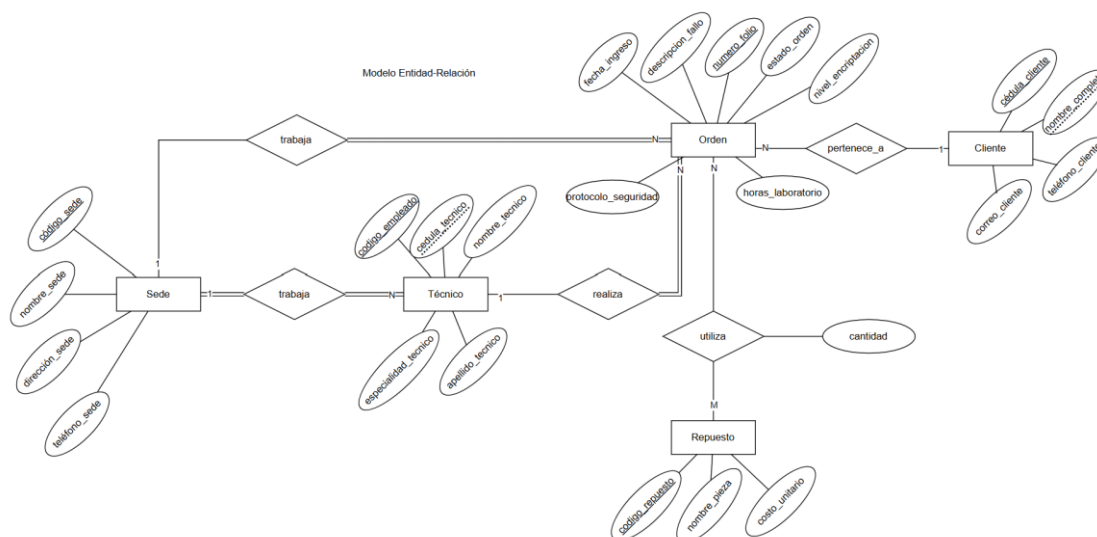
- Proceso Extra (Sede S02 - Laboratorio de Recuperación): A diferencia de la sede Norte, la Sede S02 cuenta con una certificación especial para la Recuperación de Datos Críticos. Por esta razón, solo en las órdenes generadas en la sede S02 se deben capturar datos adicionales sobre el Nivel de Encriptación del dispositivo, el Protocolo de Seguridad aplicado y las Horas de Laboratorio en cámara limpia. En la sede S01, estos registros no existen debido a que solo realizan mantenimiento preventivo y correctivo básico.

5. Control de Componentes y Repuestos

Para completar las reparaciones, se utilizan diversos Repuestos. Un repuesto puede ser utilizado en muchas órdenes de servicio diferentes a lo largo del tiempo, y una sola orden de servicio compleja puede requerir varios repuestos distintos (relación N:M). De cada repuesto se guarda un código de repuesto, el nombre de la pieza y su costo unitario.

Nota: el sistema no debe controlar un inventario de los repuestos que almacenan las sedes, sino un catálogo de los repuestos. Es decir, el registro de órdenes no implica la verificación de la existencia de repuestos enlistados en la orden.

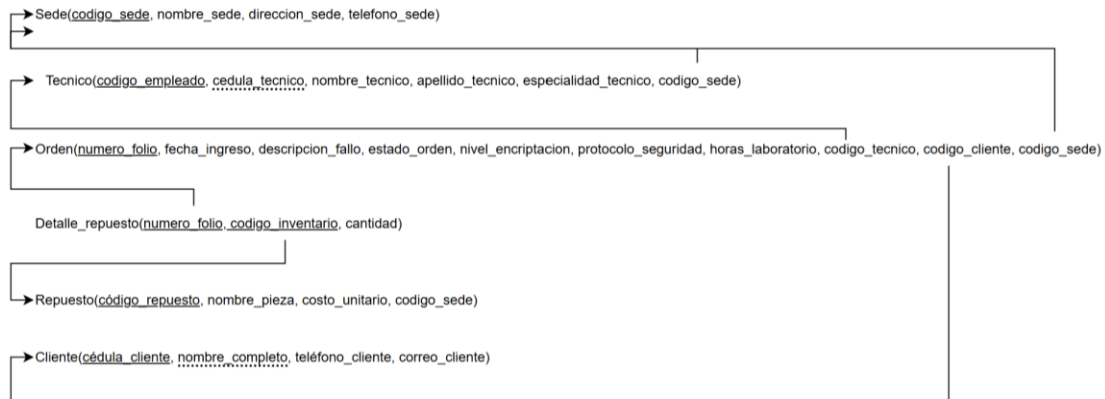
Modelo Entidad-Relación:



Grafo Relacional:



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN



Código Centralizado:

```
CREATE DATABASE TechFixCentral;
GO

USE TechFixCentral;
GO

CREATE TABLE Sede (
    codigo_sede CHAR(3) PRIMARY KEY,
    nombre_sede VARCHAR(50),
    direccion_sede VARCHAR(100),
    telefono_sede VARCHAR(20)
);

CREATE TABLE Cliente (
    cedula_cliente VARCHAR(20) PRIMARY KEY,
    nombre_completo VARCHAR(100),
    telefono_cliente VARCHAR(20),
    correo_cliente VARCHAR(100)
);

CREATE TABLE Tecnico (
    codigo_empleado INT PRIMARY KEY,
    cedula_tecnico VARCHAR(20),
    nombre_tecnico VARCHAR(50),
    apellido_tecnico VARCHAR(50),
    especialidad_tecnico VARCHAR(50),
    codigo_sede CHAR(3) FOREIGN KEY REFERENCES Sede(codigo_sede)
);

CREATE TABLE Repuesto (
    codigo_repuesto INT PRIMARY KEY,
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
nombre_pieza VARCHAR(100),
costo_unitario DECIMAL(10,2)
);

CREATE TABLE Orden (
numero_folio INT PRIMARY KEY,
fecha_ingreso DATE,
descripcion_fallo VARCHAR(255),
estado_orden VARCHAR(20),
nivel_encryptacion VARCHAR(50) NULL,
protocolo_seguridad VARCHAR(50) NULL,
horas_laboratorio INT NULL,
codigo_tecnico INT FOREIGN KEY REFERENCES
Tecnico(codigo_empleado),
cedula_cliente VARCHAR(20) FOREIGN KEY REFERENCES
Cliente(cedula_cliente),
codigo_sede CHAR(3) FOREIGN KEY REFERENCES Sede(codigo_sede)
);

CREATE TABLE Detalle_Repuesto (
numero_folio INT FOREIGN KEY REFERENCES Orden(numero_folio),
codigo_repuesto INT FOREIGN KEY REFERENCES
Repuesto(codigo_repuesto),
cantidad INT,
PRIMARY KEY (numero_folio, codigo_repuesto)
);
```

Inserción de datos centralizada:

1. SEDES

```
INSERT INTO Sede VALUES
('S01', 'Norte', 'Calle 123', '555-0101'), ('S02', 'Sur', 'Av.
Principal', '555-0202');
```

2. CLIENTES

```
INSERT INTO Cliente VALUES
('1001', 'Juan Perez', '300111', 'juan@mail.com'), ('1002',
'Maria Lopez', '300222', 'maria@mail.com'), ('1003', 'Carlos
Ruiz', '300333', 'carlos@mail.com'), ('1004', 'Ana Torres',
'300444', 'ana@mail.com'), ('1005', 'Pedro Gomez', '300555',
'pedro@mail.com'), ('1006', 'Sofia Martinez', '300666',
'sofia@mail.com'), ('1007', 'Diego Torres', '300777',
'diego@mail.com'), ('1008', 'Laura Ruiz', '300888',
'laura@mail.com'), ('1009', 'Andres Herrera', '300999',
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
'andres@mail.com'), ('1010', 'Valentina Diaz', '301010',  
'vale@mail.com'), ('1011', 'Sebastian Mora', '301111',  
'sebas@mail.com'), ('1012', 'Camila Rivas', '301212',  
'cami@mail.com'), ('1013', 'Mateo Ortega', '301313',  
'mateo@mail.com'), ('1014', 'Natalia Gil', '301414',  
'nati@mail.com'), ('1015', 'Felipe Castro', '301515',  
'felipe@mail.com');
```

3. TÉCNICOS

INSERT INTO Tecnico VALUES

```
(1, '9991', 'Luis', 'Martínez', 'Mantenimiento Preventivo',  
'S01'), (2, '9992', 'Elena', 'Vargas', 'Recuperación de Datos',  
'S02'), (3, '9993', 'Jorge', 'Castro', 'Reparación de Hardware',  
'S01'), (4, '9994', 'Pablo', 'Escobar', 'Mantenimiento  
Preventivo', 'S01'), (5, '9995', 'Lucia', 'Mendez', 'Recuperación  
de Datos', 'S02'), (6, '9996', 'Ricardo', 'Perez', 'Reparación de  
Hardware', 'S01'), (7, '9997', 'Isabel', 'Luna', 'Mantenimiento  
Preventivo', 'S01'), (8, '9998', 'Oscar', 'Vega', 'Recuperación  
de Datos', 'S02'), (9, '9999', 'Clara', 'Soler', 'Reparación de  
Hardware', 'S02'), (10, '9910', 'Hernan', 'Duarte',  
'Mantenimiento Preventivo', 'S01'), (11, '9911', 'Diana', 'Paz',  
'Recuperación de Datos', 'S02'), (12, '9912', 'Raul', 'Bermudez',  
'Reparación de Hardware', 'S01'), (13, '9913', 'Ximena', 'Rojas',  
'Recuperación de Datos', 'S02');
```

4. REPUESTOS

INSERT INTO Repuesto VALUES

```
(500, 'Pantalla LCD', 150.00), (501, 'Disco SSD 500GB', 80.00),  
(502, 'Memoria RAM 8GB', 45.00), (503, 'Cable Flex', 20.00), (504,  
'Teclado Laptop', 35.00), (505, 'Bateria 4400mAh', 55.00),  
(506, 'Cargador Universal', 25.00), (507, 'Pasta Térmica', 10.00),  
(508, 'Ventilador CPU', 20.00), (509, 'Carcasa Inferior', 40.00),  
(510, 'Touchpad', 30.00), (511, 'Modulo WiFi', 15.00), (512,  
'Altavoz Interno', 12.00), (513, 'Webcam Módulo', 22.00);
```

5. ÓRDENES

5.1 Órdenes con columnas explícitas (Sede S01)

INSERT INTO Orden (numero_folio, fecha_ingreso,
descripcion_fallo, estado_orden, codigo_tecnico, cedula_cliente,
codigo_sede)

VALUES

```
(10, '2026-05-20', 'Limpieza y cambio de pasta térmica',  
'Finalizada', 1, '1001', 'S01'),  
(11, '2026-05-25', 'Actualización de memoria', 'Pendiente', 3,  
'1003', 'S01'),
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
(30, '2026-05-30', 'Cambio de bateria', 'Pendiente', 4, '1006', 'S01'),  
(31, '2026-05-30', 'Limpieza virus', 'En Proceso', 6, '1007', 'S01'),  
(32, '2026-05-30', 'Cambio teclado', 'Pendiente', 7, '1008', 'S01'),  
(33, '2026-05-30', 'Actualización BIOS', 'Finalizada', 10, '1009', 'S01'),  
(34, '2026-05-30', 'Reparación bisagras', 'Pendiente', 12, '1010', 'S01');
```

5.2 Órdenes completas (Sede S02)

INSERT INTO Orden VALUES

```
(20, '2026-05-22', 'Recuperar archivos borrados', 'Finalizada', 'AES-256', 'Protocolo-Alpha', 12, 2, '1002', 'S02'), (21, '2026-05-28', 'Daño lógico por caída', 'En Proceso', 'RSA-2048', 'Protocolo-Beta', 8, 2, '1004', 'S02'), (22, '2026-05-29', 'Recuperación de sector defectuoso', 'Pendiente', 'AES-128', 'Protocolo-Gamma', 5, 2, '1005', 'S02'), (40, '2026-05-30', 'Recuperación disco duro', 'En Proceso', 'AES-256', 'Prot-X', 10, 5, '1011', 'S02'), (41, '2026-05-30', 'Recuperación fotos borradas', 'Finalizada', 'RSA-2048', 'Prot-Y', 4, 8, '1012', 'S02'), (42, '2026-05-30', 'Rescate partición', 'Pendiente', 'AES-128', 'Prot-Z', 15, 9, '1013', 'S02'), (43, '2026-05-30', 'Clonación disco dañado', 'En Proceso', 'AES-256', 'Prot-X', 20, 11, '1014', 'S02'), (44, '2026-05-30', 'Reparación sistema archivos', 'Pendiente', 'RSA-4096', 'Prot-W', 6, 13, '1015', 'S02');
```

6. DETALLE DE REPUESTOS

INSERT INTO Detalle_Repuesto VALUES

```
(10, 500, 1), (11, 502, 2), (20, 501, 1), (21, 503, 1), (22, 501, 1), (30, 505, 1), (31, 511, 1), (32, 504, 1), (33, 507, 1), (34, 509, 1), (40, 501, 1), (41, 501, 1), (42, 501, 1), (43, 501, 1), (44, 506, 1);
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

Diseño Distribuido:

Sedes y Roles:

Sedes	Roles
Sede Norte (S01)	Reparación y mantenimiento de hardware por órdenes
Sede Sur (S02)	Reparación y mantenimiento de hardware por órdenes/Laboratorio de recuperación de datos

Campo y Condición de fragmentación:

Campo de fragmentación = codigo_sede

Condición de fragmentación: $i = \{ 'S01', 'S02' \}$

Esquema de Fragmentación:

Vertical:

$Orden_info$

$= \prod_{numero_folio, fecha_ingreso, descripcion_fallo, estado_orden, codigo_tecnico, codigo_cliente, codigo_sede} (Orden)$

$Orden_labo = \prod_{numero_folio, nivel_encriptacion, protocolo_seguridad, horas_laboratorio} (Orden)$

Horizontal:

$Tecnico_i = \sigma_{codigo_sede = i} (Tecnico)$

$Orden_info_i = \sigma_{codigo_sede = i} (Orden_info)$

$Detalle_repuesto_i = Detalle_repuesto \bowtie Orden_info_i$

Esquema de replicación:

Tabla Sede:

1. **Justificación:** La tabla presenta baja transaccionalidad y se usa para consulta.
2. **Nodos involucrados:** Sede S01, Sede S02
3. **Tipo de replicación:** Bidireccional
4. **Nodo de gestión:** -

Tabla Cliente:



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

1. **Justificación:** Debido a que los clientes pueden realizar órdenes en ambas sedes, se requiere que su información esté disponible en ambas sedes de forma inmediata.
2. **Nodos involucrados:** Sede S01, Sede S02
3. **Tipo de replicación:** Bidireccional
4. **Nodo de gestión:** -

Tabla Repuesto:

1. **Justificación:** La tabla Repuesto mantiene baja transaccionalidad, y la tabla se considera una tabla de catálogo para las órdenes de ambas sedes.
2. **Nodos involucrados:** Sede S01, Sede S02
3. **Tipo de replicación:** Bidireccional
4. **Nodo de gestión:** -

Esquema de ubicación/asignación:

	Sede Norte S01	Sede Sur S02
Sede	Sede	Sede
Tecnico	Tecnico _{S01}	Tecnico _{S02}
Orden	Orden_info _{S01}	Orden_info _{S02} Orden_labo
Detalle_repuesto	Detalle_repuesto _{S01}	Detalle_repuesto _{S02}
Repuesto	Repuesto	Repuesto
Cliente	Cliente	Cliente

Código Sede Norte:

```
USE TechFixS01;
GO

/*-----*/
-- TABLAS FRAGMENTADAS
/*-----*/

create table TecnicoS01
(
    codigo_empleado int not null,
    cedula_tecnico varchar(20),
    nombre_tecnico varchar(50),
    apellido_tecnico varchar(50),
    especialidad_tecnico varchar(50),
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
        codigo_sede char(3) not null
    )

create table Orden_infoS01
(
    numero_folio int not null,
    fecha_ingreso date,
    descripcion_fallo varchar(255),
    estado_orden varchar(20),
    codigo_tecnico int,
    cedula_cliente varchar(20),
    codigo_sede char(3) not null
)

/* Fragmentación Horizontal Derivada */
create table Detalle_RepuestoS01
(
    numero_folio int not null,
    codigo_repuesto int not null,
    cantidad int,
    codigo_sede char(3) not null
)

/*-----*/
-- CREACIÓN DE PKs Y FKs
/*-----*/

-- TecnicoS01
alter table TecnicoS01 add constraint
pk_tecnicoS01 primary key (codigo_empleado, codigo_sede)

alter table TecnicoS01 add constraint
fk_sede_tecnicoS01 foreign key (codigo_sede)
references Sede(codigo_sede)

-- Orden_infoS01
alter table Orden_infoS01 add constraint
pk_orden_infoS01 primary key (numero_folio, codigo_sede)

alter table Orden_infoS01 add constraint
fk_tecnico_infoS01 foreign key (codigo_tecnico, codigo_sede)
references TecnicoS01(codigo_empleado, codigo_sede)

alter table Orden_infoS01 add constraint
fk_cliente_infoS01 foreign key (cedula_cliente)
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
references Cliente(cedula_cliente)

alter table Orden_infoS01 add constraint
fk_sede_infoS01 foreign key (codigo_sede)
references Sede(codigo_sede)

-- Detalle_RepuestoS01
alter table Detalle_RepuestoS01 add constraint
pk_detalle_repuestoS01 primary key (numero_folio,
codigo_repuesto, codigo_sede)

alter table Detalle_RepuestoS01 add constraint
fk_orden_detalleS01 foreign key (numero_folio, codigo_sede)
references Orden_infoS01(numero_folio, codigo_sede)

alter table Detalle_RepuestoS01 add constraint
fk_repuesto_detalleS01 foreign key (codigo_repuesto)
references Repuesto(codigo_repuesto)

--- Checks para S01
ALTER TABLE TecnicoS01
ADD CONSTRAINT ck_tecnicoS01_sede CHECK (codigo_sede = 'S01');

ALTER TABLE Orden_infoS01
ADD CONSTRAINT ck_orden_infoS01_sede CHECK (codigo_sede = 'S01');

ALTER TABLE Detalle_RepuestoS01
ADD CONSTRAINT ck_detalle_repuestoS01_sede CHECK (codigo_sede =
'S01');

/*-----*/
-- RECUPERAR DATOS DESDE NODO REMOTO
/*-----*/

-- Tabla TecnicoS01
insert into TechFixS01.dbo.TecnicoS01
select codigo_empleado, cedula_tecnico, nombre_tecnico,
apellido_tecnico, especialidad_tecnico, codigo_sede
from [26.226.35.148].[TechFixCentral].[dbo].[Tecnico]
where codigo_sede = 'S01'

-- Tabla Orden_infoS01
insert into TechFixS01.dbo.Orden_infoS01
select numero_folio, fecha_ingreso, descripcion_fallo,
estado_orden, codigo_tecnico, cedula_cliente, codigo_sede
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
from [26.226.35.148].[TechFixCentral].[dbo].[Orden]
where codigo_sede = 'S01'

-- Tabla Detalle_RepuestoS01 (Fragmentación Derivada)
insert into TechFixS01.dbo.Detalle_RepuestoS01
select      d.numero_folio,      d.codigo_repuesto,      d.cantidad,
o.codigo_sede
from [26.226.35.148].[TechFixCentral].[dbo].[Detalle_Repuesto] d
join  [26.226.35.148].[TechFixCentral].[dbo].[Orden]      o      on
d.numero_folio = o.numero_folio
where o.codigo_sede = 'S01'

select * from TecnicoS01
select * from Orden_infoS01
select * from Detalle_RepuestoS01
```

Código Sede Sur S02:

```
CREATE DATABASE TechFixS02 COLLATE Modern_Spanish_CI_AS;
GO

USE TechFixS02;
GO

/*-----*/
-- TABLAS REPLICADAS
/*-----*/

CREATE TABLE Sede (
    codigo_sede CHAR(3) NOT NULL,
    nombre_sede VARCHAR(50),
    direccion_sede VARCHAR(100),
    telefono_sede VARCHAR(20)
);

CREATE TABLE Cliente (
    cedula_cliente VARCHAR(20) NOT NULL,
    nombre_completo VARCHAR(100),
    telefono_cliente VARCHAR(20),
    correo_cliente VARCHAR(100)
);

CREATE TABLE Repuesto (
    codigo_repuesto INT NOT NULL,
    nombre_pieza VARCHAR(100),
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
costo_unitario DECIMAL(10,2)
);

/*-----*/
-- TABLAS FRAGMENTADAS
/*-----*/

-- Fragmentación Horizontal
CREATE TABLE Tecnicos02 (
    codigo_empleado INT NOT NULL,
    cedula_tecnico VARCHAR(20),
    nombre_tecnico VARCHAR(50),
    apellido_tecnico VARCHAR(50),
    especialidad_tecnico VARCHAR(50),
    codigo_sede CHAR(3) NOT NULL
);

-- Fragmentación Vertical
CREATE TABLE Orden_labo (
    numero_folio INT NOT NULL,
    nivel_encryption VARCHAR(50) NULL,
    protocolo_seguridad VARCHAR(50) NULL,
    horas_laboratorio INT NULL
);

-- Fragmentación Horizontal
CREATE TABLE Orden_info02 (
    numero_folio INT NOT NULL,
    fecha_ingreso DATE,
    descripcion_fallo VARCHAR(255),
    estado_orden VARCHAR(20),
    codigo_tecnico INT,
    cedula_cliente VARCHAR(20),
    codigo_sede CHAR(3) NOT NULL
);

-- Fragmentación Horizontal Derivada
CREATE TABLE Detalle_Repuesto02 (
    numero_folio INT NOT NULL,
    codigo_repuesto INT NOT NULL,
    cantidad INT,
    codigo_sede CHAR(3) NOT NULL
);

/*-----*/
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
-- CREACIÓN DE PKs Y FKs
/*-----*/

-- Sede
ALTER TABLE Sede ADD CONSTRAINT
pk_sede PRIMARY KEY (codigo_sede);

-- Cliente
ALTER TABLE Cliente ADD CONSTRAINT
pk_cliente PRIMARY KEY (cedula_cliente);

-- Repuesto
ALTER TABLE Repuesto ADD CONSTRAINT
pk_repuesto PRIMARY KEY (codigo_repuesto);

-- TecnicoS02
ALTER TABLE TecnicoS02 ADD CONSTRAINT
pk_tecnicoS02 PRIMARY KEY (codigo_empleado, codigo_sede);

ALTER TABLE TecnicoS02 ADD CONSTRAINT
fk_sede_tecnicoS02 FOREIGN KEY (codigo_sede)
REFERENCES Sede(codigo_sede);

-- Orden_labo
ALTER TABLE Orden_labo ADD CONSTRAINT
pk_orden_labo PRIMARY KEY (numero_folio);

-- Orden_infoS02
ALTER TABLE Orden_infoS02 ADD CONSTRAINT
pk_orden_infoS02 PRIMARY KEY (numero_folio, codigo_sede);

ALTER TABLE Orden_infoS02 ADD CONSTRAINT
fk_tecnico_infoS02 FOREIGN KEY (codigo_tecnico, codigo_sede)
REFERENCES TecnicoS02(codigo_empleado, codigo_sede);

ALTER TABLE Orden_infoS02 ADD CONSTRAINT fk_orden_labo_infoS02
FOREIGN KEY (numero_folio) REFERENCES Orden_labo(numero_folio);

ALTER TABLE Orden_infoS02 ADD CONSTRAINT
fk_cliente_infoS02 FOREIGN KEY (cedula_cliente)
REFERENCES Cliente(cedula_cliente);

ALTER TABLE Orden_infoS02 ADD CONSTRAINT
fk_sede_infoS02 FOREIGN KEY (codigo_sede)
REFERENCES Sede(codigo_sede);
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
-- Detalle_RepuestoS02
ALTER TABLE Detalle_RepuestoS02 ADD CONSTRAINT
pk_detalle_repuestoS02 PRIMARY KEY (numero_folio,
codigo_repuesto, codigo_sede);

ALTER TABLE Detalle_RepuestoS02 ADD CONSTRAINT
fk_orden_detalleS02 FOREIGN KEY (numero_folio, codigo_sede)
REFERENCES Orden_infoS02(numero_folio, codigo_sede);

ALTER TABLE Detalle_RepuestoS02 ADD CONSTRAINT
fk_repuesto_detalleS02 FOREIGN KEY (codigo_repuesto)
REFERENCES Repuesto(codigo_repuesto);

--- Checks para S02

ALTER TABLE TecnicoS02
ADD CONSTRAINT ck_tecnicoS02_sede CHECK (codigo_sede = 'S02');

ALTER TABLE Orden_infoS02
ADD CONSTRAINT ck_orden_infoS02_sede CHECK (codigo_sede =
'S02');

ALTER TABLE Detalle_RepuestoS02
ADD CONSTRAINT ck_detalle_repuestoS02_sede CHECK (codigo_sede =
'S02');

/*-----*/
-- RECUPERAR DATOS DE LA BASE CENTRALIZADA
/*-----*/

-- Tabla Sede
INSERT INTO TechFixS02.dbo.Sede
SELECT * FROM TechFixCentral.dbo.Sede;

-- Tabla Cliente
INSERT INTO TechFixS02.dbo.Cliente
SELECT * FROM TechFixCentral.dbo.Cliente;

-- Tabla Repuesto
INSERT INTO TechFixS02.dbo.Repuesto
SELECT * FROM TechFixCentral.dbo.Repuesto;

-- Tabla TecnicoS02
INSERT INTO TechFixS02.dbo.TecnicoS02
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
SELECT    codigo_empleado,    cedula_tecnico,    nombre_tecnico,
apellido_tecnico, especialidad_tecnico, codigo_sede
FROM TechFixCentral.dbo.Tecnico
WHERE codigo_sede = 'S02';

-- Tabla Orden_labo
INSERT INTO TechFixS02.dbo.Orden_labo
SELECT    numero_folio,    nivel_encryptacion,    protocolo_seguridad,
horas_laboratorio
FROM TechFixCentral.dbo.Orden;

-- Tabla Orden_infoS02
INSERT INTO TechFixS02.dbo.Orden_infoS02
SELECT    numero_folio,    fecha_ingreso,    descripcion_fallo,
estado_orden, codigo_tecnico, cedula_cliente, codigo_sede
FROM TechFixCentral.dbo.Orden
WHERE codigo_sede = 'S02';

-- Tabla Detalle_RepuestoS02
INSERT INTO TechFixS02.dbo.Detalle_RepuestoS02
SELECT    d.numero_folio,    d.codigo_repuesto,    d.cantidad,
o.codigo_sede
FROM TechFixCentral.dbo.Detalle_Repuesto d
JOIN    TechFixCentral.dbo.Orden    o    ON    d.numero_folio    =
o.numero_folio
WHERE o.codigo_sede = 'S02';

/*-----*/
-- LISTAR TODOS LOS REGISTROS VERIFICACIÓN
/*-----*/

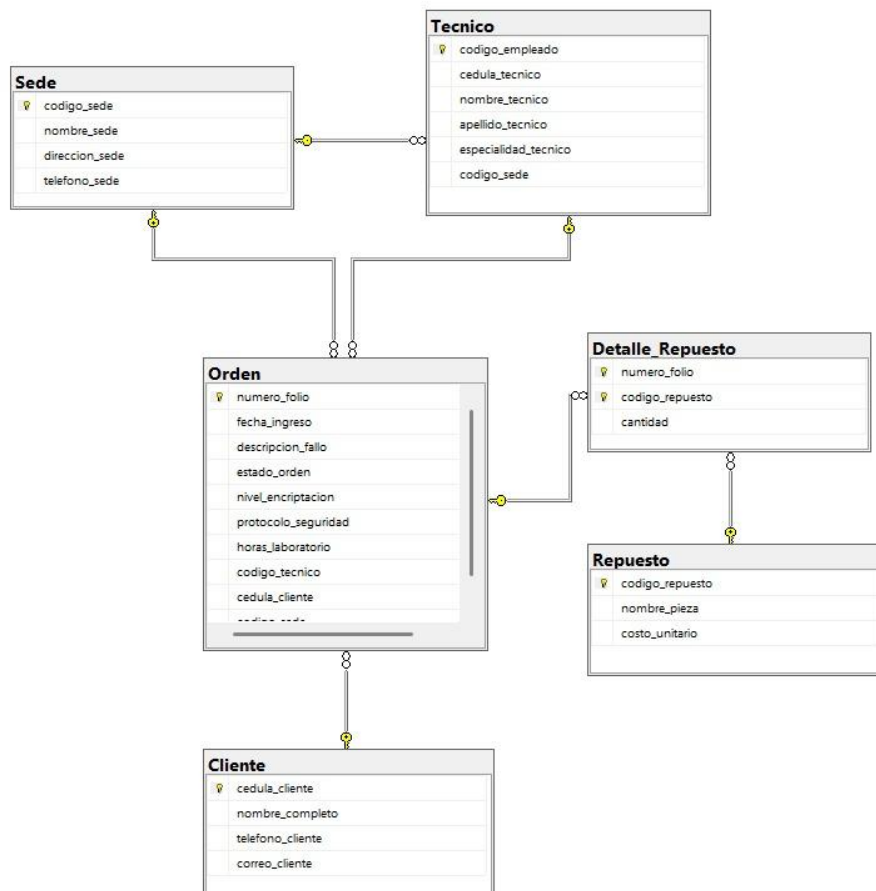
SELECT * FROM Sede;
SELECT * FROM Cliente;
SELECT * FROM Repuesto;
SELECT * FROM TecnicoS02;
SELECT * FROM Orden_labo;
SELECT * FROM Orden_infoS02;
SELECT * FROM Detalle_RepuestoS02;
```

Esquemas físicos de los nodos:

Base centralizada:



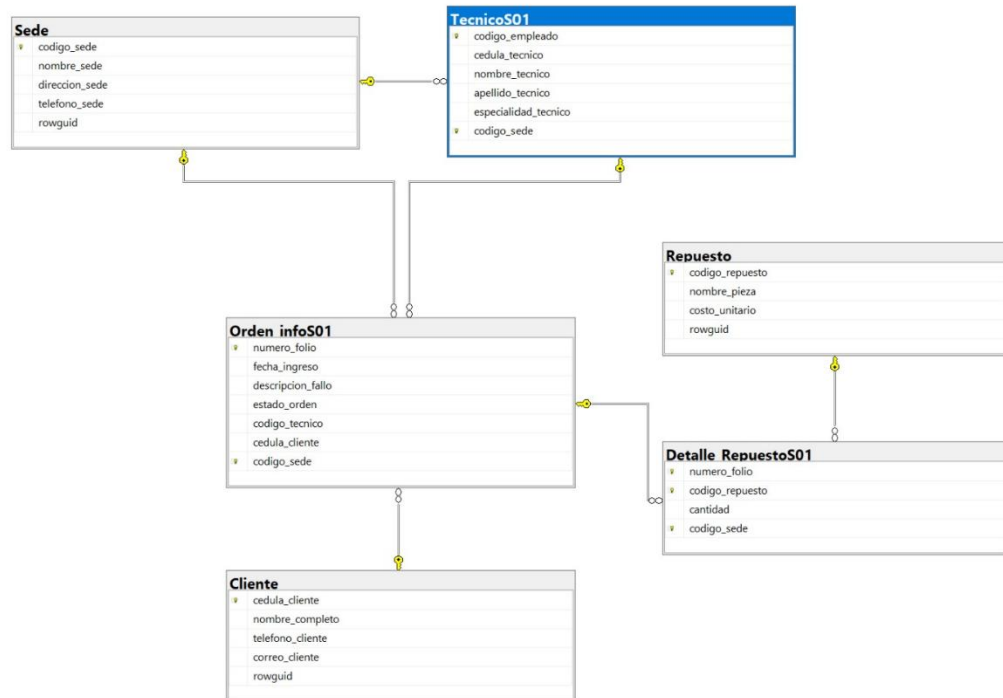
ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN



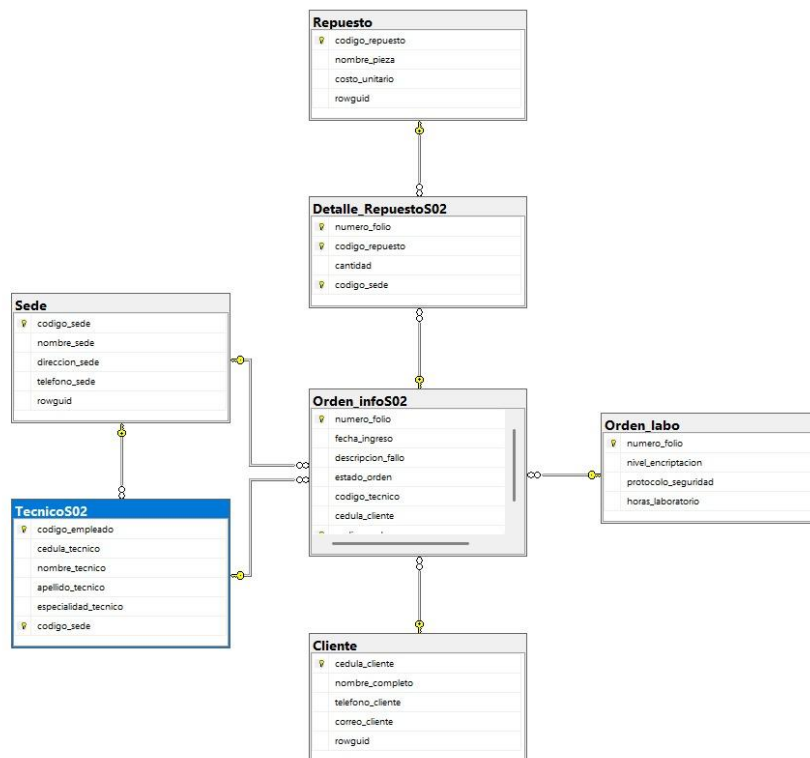
Sede S01:



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN



Sede S02:





ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

Vistas distribuidas actualizables:

```
/* =====
1. VISTAS DISTRIBUIDAS - NODO S02
=====
*/
CREATE VIEW VDA_Tecnico AS
SELECT codigo_empleado, cedula_tecnico, nombre_tecnico,
apellido_tecnico, especialidad_tecnico, codigo_sede
FROM TechFixS02.dbo.TecnicoS02
UNION ALL
SELECT codigo_empleado, cedula_tecnico, nombre_tecnico,
apellido_tecnico, especialidad_tecnico, codigo_sede
FROM [26.239.167.8].[TechFixS01].[dbo].[TecnicoS01];
GO

CREATE VIEW VDA_Orden_info AS
SELECT numero_folio, fecha_ingreso, descripcion_fallo, estado_orden,
codigo_tecnico, cedula_cliente, codigo_sede
FROM TechFixS02.dbo.Orden_infoS02
UNION ALL
SELECT numero_folio, fecha_ingreso, descripcion_fallo, estado_orden,
codigo_tecnico, cedula_cliente, codigo_sede
FROM [26.239.167.8].[TechFixS01].[dbo].[Orden_infoS01];
GO

CREATE VIEW VDA_Detalle_Repuesto AS
SELECT numero_folio, codigo_repuesto, cantidad, codigo_sede
FROM TechFixS02.dbo.Detalle_RepuestoS02
UNION ALL
SELECT numero_folio, codigo_repuesto, cantidad, codigo_sede
FROM [26.239.167.8].[TechFixS01].[dbo].[Detalle_RepuestoS01];
GO

/* =====
1. VISTAS DISTRIBUIDAS - NODO S01
=====
*/
CREATE VIEW VDA_Tecnico AS
SELECT codigo_empleado, cedula_tecnico, nombre_tecnico,
apellido_tecnico, especialidad_tecnico, codigo_sede
FROM TechFixS01.dbo.TecnicoS01
UNION ALL
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
SELECT codigo_empleado, cedula_tecnico, nombre_tecnico,
apellido_tecnico, especialidad_tecnico, codigo_sede
FROM [26.226.35.148].[TechFixS02].[dbo].[TecnicoS02];
GO

CREATE VIEW VDA_Orden_info AS
SELECT numero_folio, fecha_ingreso, descripcion_fallo, estado_orden,
codigo_tecnico, cedula_cliente, codigo_sede
FROM TechFixS01.dbo.Orden_infoS01
UNION ALL
SELECT numero_folio, fecha_ingreso, descripcion_fallo, estado_orden,
codigo_tecnico, cedula_cliente, codigo_sede
FROM [26.226.35.148].[TechFixS02].[dbo].[Orden_infoS02];
GO

CREATE VIEW VDA_Detalle_Repuesto AS
SELECT numero_folio, codigo_repuesto, cantidad, codigo_sede
FROM TechFixS01.dbo.Detalle_RepuestoS01
UNION ALL
SELECT numero_folio, codigo_repuesto, cantidad, codigo_sede
FROM [26.226.35.148].[TechFixS02].[dbo].[Detalle_RepuestoS02];
GO
```

Procedimientos almacenados:

```
/*-----*/
-- 1. LECTURA PARA DASHBOARD
/*-----*/
GO

CREATE PROCEDURE sp_ObtenerDashboardOrdenes
    @sede_filtro CHAR(3),
    @estado_filtro VARCHAR(20) = NULL,
    @fecha_busqueda DATE = NULL
AS BEGIN
SET NOCOUNT ON;
    SELECT
        o.numero_folio,
        o.fecha_ingreso,
        o.descripcion_fallo,
        o.estado_orden,
        t.nombre_tecnico + ' ' + t.apellido_tecnico AS nombre_tecnico,
        c.nombre_completo AS nombre_cliente,
        o.codigo_sede,
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
        l.nivel_encryption,
        l.protocolo_seguridad,
        l.horas_laboratorio
FROM VDA_Orden_info o
LEFT JOIN VDA_Tecnico t ON o.codigo_tecnico = t.codigo_empleado
LEFT JOIN Cliente c ON o.cedula_cliente = c.cedula_cliente
-- S01 consulta a S02 remotamente
LEFT JOIN [26.226.35.148].[TechFixS02].[dbo].[Orden_labo] l ON
o.numero_folio = l.numero_folio
WHERE
    o.codigo_sede = @sede_filtro AND
    (@estado_filtro IS NULL OR @estado_filtro = '' OR
o.estado_orden = @estado_filtro) AND
    (@fecha_busqueda IS NULL OR o.fecha_ingreso = @fecha_busqueda)
ORDER BY o.fecha_ingreso DESC, o.numero_folio DESC;
END;
GO

/*-----*/
-- 2. GESTIÓN DE CLIENTES
/*-----*/
CREATE PROCEDURE sp_InsertarCliente
    @cedula VARCHAR(20), @nombre VARCHAR(100), @telefono VARCHAR(20),
@correo VARCHAR(100)
AS BEGIN
SET NOCOUNT ON;
    BEGIN TRY
        IF LTRIM(RTRIM(ISNULL(@cedula, ''))) = '' THROW 50001, 'Error:
La cédula no puede estar vacía.', 1;
        IF LTRIM(RTRIM(ISNULL(@nombre, ''))) = '' THROW 50002, 'Error:
El nombre no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@telefono, ''))) = '' THROW 50003,
'Error: El teléfono no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@correo, ''))) = '' THROW 50004, 'Error:
El correo no puede estar vacío.', 1;

        IF LEN(@cedula) < 10 THROW 50005, 'Error: La cédula debe tener
al menos 10 caracteres.', 1;
        IF @correo NOT LIKE '%@__%.__%' THROW 50006, 'Error: Formato
de correo electrónico inválido.', 1;
        IF EXISTS (SELECT 1 FROM Cliente WHERE cedula_cliente =
@cedula) THROW 50007, 'Error: El cliente ya existe en el sistema.', 1;
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
BEGIN DISTRIBUTED TRANSACTION;
INSERT INTO Cliente (cedula_cliente, nombre_completo,
telefono_cliente, correo_cliente)
VALUES (@cedula, @nombre, @telefono, @correo);
COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH
END;
GO

CREATE PROCEDURE sp_ActualizarCliente
    @cedula VARCHAR(20), @nombre VARCHAR(100), @telefono VARCHAR(20),
@correo VARCHAR(100)
AS BEGIN
SET NOCOUNT ON;
BEGIN TRY
    IF LTRIM(RTRIM(ISNULL(@cedula, ''))) = '' THROW 50001, 'Error:
La cédula no puede estar vacía.', 1;
    IF LTRIM(RTRIM(ISNULL(@nombre, ''))) = '' THROW 50002, 'Error:
El nombre no puede estar vacío.', 1;
    IF LTRIM(RTRIM(ISNULL(@telefono, ''))) = '' THROW 50003,
'Error: El teléfono no puede estar vacío.', 1;
    IF LTRIM(RTRIM(ISNULL(@correo, ''))) = '' THROW 50004, 'Error:
El correo no puede estar vacío.', 1;

    IF NOT EXISTS (SELECT 1 FROM Cliente WHERE cedula_cliente =
@cedula) THROW 50008, 'Error: El cliente no existe.', 1;
    IF @correo NOT LIKE '%@__%.__%' THROW 50006, 'Error: Formato
de correo electrónico inválido.', 1;

    BEGIN DISTRIBUTED TRANSACTION;
    UPDATE Cliente
    SET nombre_completo = @nombre, telefono_cliente = @telefono,
correo_cliente = @correo
    WHERE cedula_cliente = @cedula;
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
        END CATCH
    END;
GO

CREATE PROCEDURE sp_EliminarCliente
    @cedula VARCHAR(20)
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF LTRIM(RTRIM(ISNULL(@cedula, ''))) = '' THROW 50001, 'Error:
La cédula no puede estar vacía.', 1;
        IF EXISTS (SELECT 1 FROM VDA_Orden_info WHERE cedula_cliente =
@cedula)
            THROW 50009, 'Error: No se puede eliminar el cliente
porque tiene órdenes de servicio asociadas.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        DELETE FROM Cliente WHERE cedula_cliente = @cedula;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

/*-----*/
-- 3. GESTIÓN DE TÉCNICOS
/*-----*/

CREATE PROCEDURE sp_InsertarTecnico
    @cedula VARCHAR(20), @nombre VARCHAR(50), @apellido VARCHAR(50),
@especialidad VARCHAR(50), @sede CHAR(3)
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF LTRIM(RTRIM(ISNULL(@cedula, ''))) = '' THROW 50010, 'Error:
La cédula del técnico no puede estar vacía.', 1;
        IF LTRIM(RTRIM(ISNULL(@nombre, ''))) = '' THROW 50011, 'Error:
El nombre no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@apellido, ''))) = '' THROW 50012,
'Error: El apellido no puede estar vacío.', 1;
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
        IF LTRIM(RTRIM(ISNULL(@especialidad, ''))) = '' THROW 50013,
'Error: La especialidad no puede estar vacía.', 1;
        IF LTRIM(RTRIM(ISNULL(@sede, ''))) = '' THROW 50014, 'Error:
La sede no puede estar vacía.', 1;

        IF @sede NOT IN ('S01', 'S02') THROW 50015, 'Error: La sede
debe ser S01 o S02.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        DECLARE @nuevo_codigo INT;
        SELECT @nuevo_codigo = ISNULL(MAX(codigo_empleado), 0) + 1
FROM VDA_Tecnico;

        INSERT INTO VDA_Tecnico (codigo_empleado, cedula_tecnico,
nombre_tecnico, apellido_tecnico, especialidad_tecnico, codigo_sede)
VALUES (@nuevo_codigo, @cedula, @nombre, @apellido,
@especialidad, @sede);
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

CREATE PROCEDURE sp_ActualizarTecnico
    @codigo INT, @cedula VARCHAR(20), @nombre VARCHAR(50), @apellido
VARCHAR(50), @especialidad VARCHAR(50)
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF @codigo IS NULL THROW 50016, 'Error: El código de empleado
no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@cedula, ''))) = '' THROW 50010, 'Error:
La cédula del técnico no puede estar vacía.', 1;
        IF LTRIM(RTRIM(ISNULL(@nombre, ''))) = '' THROW 50011, 'Error:
El nombre no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@apellido, ''))) = '' THROW 50012,
'Error: El apellido no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@especialidad, ''))) = '' THROW 50013,
'Error: La especialidad no puede estar vacía.', 1;
```




ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
        IF NOT EXISTS (SELECT 1 FROM VDA_Tecnico WHERE codigo_empleado
= @codigo) THROW 50017, 'Error: El técnico no existe.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        UPDATE VDA_Tecnico
        SET cedula_tecnico = @cedula, nombre_tecnico = @nombre,
            apellido_tecnico = @apellido, especialidad_tecnico =
@especialidad
        WHERE codigo_empleado = @codigo;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

CREATE PROCEDURE sp_EliminarTecnico
    @codigo INT
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF @codigo IS NULL THROW 50016, 'Error: El código de empleado
no puede estar vacío.', 1;
        IF EXISTS (SELECT 1 FROM VDA_Orden_info WHERE codigo_tecnico =
@codigo)
            THROW 50018, 'Error: No se puede eliminar el técnico
porque tiene órdenes asignadas.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        DELETE FROM VDA_Tecnico WHERE codigo_empleado = @codigo;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

/*-----*/
-- 4. GESTIÓN DE REPUESTOS
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
/*-----*/
CREATE PROCEDURE sp_InsertarRepuesto
    @nombre VARCHAR(100), @costo DECIMAL(10,2)
AS BEGIN
SET NOCOUNT ON;
    BEGIN TRY
        IF LTRIM(RTRIM(ISNULL(@nombre, ''))) = '' THROW 50019, 'Error:
El nombre de la pieza no puede estar vacío.', 1;
        IF @costo IS NULL OR @costo < 0 THROW 50020, 'Error: El costo
unitario debe ser válido y no negativo.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        DECLARE @nuevo_codigo INT;
        SELECT @nuevo_codigo = ISNULL(MAX(codigo_repuesto), 0) + 1
FROM Repuesto;

        INSERT INTO Repuesto (codigo_repuesto, nombre_pieza,
costo_unitario)
        VALUES (@nuevo_codigo, @nombre, @costo);
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

CREATE PROCEDURE sp_ActualizarRepuesto
    @codigo INT, @nombre VARCHAR(100), @costo DECIMAL(10,2)
AS BEGIN
SET NOCOUNT ON;
    BEGIN TRY
        IF @codigo IS NULL THROW 50021, 'Error: El código de repuesto
no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@nombre, ''))) = '' THROW 50019, 'Error:
El nombre de la pieza no puede estar vacío.', 1;
        IF @costo IS NULL OR @costo < 0 THROW 50020, 'Error: El costo
unitario debe ser válido y no negativo.', 1;
        IF NOT EXISTS (SELECT 1 FROM Repuesto WHERE codigo_repuesto =
@codigo) THROW 50022, 'Error: El repuesto no existe.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
        UPDATE Repuesto
        SET nombre_pieza = @nombre, costo_unitario = @costo
        WHERE codigo_repuesto = @codigo;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

CREATE PROCEDURE sp_EliminarRepuesto
    @codigo INT
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF @codigo IS NULL THROW 50021, 'Error: El código de repuesto
no puede estar vacío.', 1;
        IF EXISTS (SELECT 1 FROM VDA_Detalle_Repuesto WHERE
codigo_repuesto = @codigo)
            THROW 50023, 'Error: No se puede eliminar el repuesto
porque está asignado a una orden.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        DELETE FROM Repuesto WHERE codigo_repuesto = @codigo;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

/*-----*/
-- 5. GESTIÓN DE ÓRDENES - NODO S01
/*-----*/
CREATE PROCEDURE sp_InsertarOrden
    @fecha DATE, @fallo VARCHAR(255), @estado VARCHAR(20), @tecnico
INT, @cliente VARCHAR(20), @sede CHAR(3),
    @encriptacion VARCHAR(50) = NULL, @protocolo VARCHAR(50) = NULL,
@horas INT = NULL
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF LTRIM(RTRIM(ISNULL(@fallo, ''))) = '' THROW 50024, 'Error:
La descripción del fallo no puede estar vacía.', 1;
        IF LTRIM(RTRIM(ISNULL(@estado, ''))) = '' THROW 50025, 'Error:
El estado de la orden no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@cliente, ''))) = '' THROW 50026,
'Error: La cédula del cliente no puede estar vacía.', 1;
        IF LTRIM(RTRIM(ISNULL(@sede, ''))) = '' THROW 50027, 'Error:
La sede no puede estar vacía.', 1;
        IF @tecnico IS NULL THROW 50028, 'Error: Debe asignar un
técnico válido.', 1;
        IF @fecha IS NULL SET @fecha = GETDATE();

        IF @estado NOT IN ('Pendiente', 'En Proceso', 'Finalizada')
THROW 50029, 'Error: Estado de orden inválido.', 1;
        IF @fecha > GETDATE() THROW 50030, 'Error: La fecha de ingreso
no puede ser futura.', 1;
        IF NOT EXISTS (SELECT 1 FROM Cliente WHERE cedula_cliente =
@cliente) THROW 50031, 'Error: El cliente no existe.', 1;

        DECLARE @sede_tecnico CHAR(3);
        SELECT @sede_tecnico = codigo_sede FROM VDA_Tecnico WHERE
codigo_empleado = @tecnico;
        IF @sede_tecnico IS NULL THROW 50032, 'Error: El técnico no
existe.', 1;
        IF @sede_tecnico <> @sede THROW 50040, 'Error: El técnico
asignado no pertenece a la sede de la orden.', 1;

        BEGIN DISTRIBUTED TRANSACTION;

        DECLARE @nuevo_folio INT;
        IF @sede = 'S01' BEGIN
            SELECT @nuevo_folio = ISNULL(MAX(numero_folio), 99) FROM
VDA_Orden_info WHERE numero_folio % 2 <> 0 AND numero_folio >= 101;
            SET @nuevo_folio = @nuevo_folio + 2;

            INSERT INTO VDA_Orden_info (numero_folio, fecha_ingreso,
descripcion_fallo, estado_orden, codigo_tecnico, cedula_cliente,
codigo_sede)
                VALUES (@nuevo_folio, @fecha, @fallo, @estado, @tecnico,
@cliente, @sede);
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
END
ELSE IF @sede = 'S02' BEGIN
    SELECT @nuevo_folio = ISNULL(MAX(numero_folio), 98) FROM
VDA_Orden_info WHERE numero_folio % 2 = 0 AND numero_folio >= 100;
    SET @nuevo_folio = @nuevo_folio + 2;

    -- ORDEN DE INSERCIÓN CORREGIDO (Vía VPN Linked Server)
    INSERT INTO
[26.226.35.148].[TechFixS02].[dbo].[Orden_labo] (numero_folio,
nivel_encryptacion, protocolo_seguridad, horas_laboratorio)
    VALUES (@nuevo_folio, @encryptacion, @protocolo, @horas);

    INSERT INTO VDA_Orden_info (numero_folio, fecha_ingreso,
descripcion_fallo, estado_orden, codigo_tecnico, cedula_cliente,
codigo_sede)
    VALUES (@nuevo_folio, @fecha, @fallo, @estado, @tecnico,
@cliente, @sede);
END
ELSE BEGIN
    THROW 50033, 'Error: Sede no reconocida.', 1;
END
SELECT @nuevo_folio AS folio_generado;
COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH
END;
GO

CREATE PROCEDURE sp_ActualizarOrden
    @folio INT, @fallo VARCHAR(255), @estado VARCHAR(20), @tecnico
INT, @encryptacion VARCHAR(50) = NULL, @protocolo VARCHAR(50) = NULL,
@horas INT = NULL
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF @folio IS NULL THROW 50034, 'Error: El folio de la orden no
puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@fallo, ''))) = '' THROW 50024, 'Error:
La descripción del fallo no puede estar vacía.', 1;
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
        IF LTRIM(RTRIM(ISNULL(@estado, ''))) = '' THROW 50025, 'Error:
El estado de la orden no puede estar vacío.', 1;
        IF @tecnico IS NULL THROW 50028, 'Error: Debe asignar un
técnico válido.', 1;
        IF @estado NOT IN ('Pendiente', 'En Proceso', 'Finalizada')
THROW 50029, 'Error: Estado de orden inválido.', 1;

        DECLARE @sede CHAR(3);
        SELECT @sede = codigo_sede FROM VDA_Orden_info WHERE
numero_folio = @folio;
        IF @sede IS NULL THROW 50035, 'Error: La orden no existe.', 1;

        DECLARE @sede_tecnico CHAR(3);
        SELECT @sede_tecnico = codigo_sede FROM VDA_Tecnico WHERE
codigo_empleado = @tecnico;
        IF @sede_tecnico IS NULL THROW 50032, 'Error: El técnico no
existe.', 1;
        IF @sede_tecnico <> @sede THROW 50040, 'Error: El técnico
asignado no pertenece a la sede de la orden.', 1;

        BEGIN DISTRIBUTED TRANSACTION;

        UPDATE VDA_Orden_info
        SET descripcion_fallo = @fallo, estado_orden = @estado,
codigo_tecnico = @tecnico
        WHERE numero_folio = @folio;

        IF @sede = 'S02' BEGIN
            UPDATE [26.226.35.148].[TechFixS02].[dbo].[Orden_labo]
            SET nivel_encryptacion = @encryptacion,
protocolo_seguridad = @protocolo, horas_laboratorio = @horas
            WHERE numero_folio = @folio;
        END
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

CREATE PROCEDURE sp_EliminarOrden
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
@folio INT
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF @folio IS NULL THROW 50034, 'Error: El folio de la orden no
puede estar vacío.', 1;

        DECLARE @sede CHAR(3);
        DECLARE @estadoActual VARCHAR(20);

        SELECT @sede = codigo_sede, @estadoActual = estado_orden
        FROM VDA_Orden_info WHERE numero_folio = @folio;

        IF @sede IS NULL THROW 50035, 'Error: La orden no existe.', 1;
        IF @estadoActual <> 'Pendiente'
            THROW 50036, 'Error: Acción denegada. Solo se pueden
eliminar órdenes que se encuentren en estado "Pendiente".', 1;

        BEGIN DISTRIBUTED TRANSACTION;

        -- ORDEN DE ELIMINACIÓN CORREGIDO (Vía VPN Linked Server)
        -- 1. Se borran los repuestos
        DELETE FROM VDA_Detalle_Repuesto WHERE numero_folio = @folio;

        -- 2. Se borra la Orden general (Hijo de Orden_labo)
        DELETE FROM VDA_Orden_info WHERE numero_folio = @folio;

        -- 3. Se borra el laboratorio (Padre)
        IF @sede = 'S02' BEGIN
            DELETE FROM
[26.226.35.148].[TechFixS02].[dbo].[Orden_labo] WHERE numero_folio =
@folio;
        END

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
/*-----*/
-- 6. GESTIÓN DE DETALLES DE REPUESTO
/*-----*/
CREATE PROCEDURE sp_InsertarDetalleRepuesto
    @folio INT, @codigo_repuesto INT, @cantidad INT
AS BEGIN
SET NOCOUNT ON;
    BEGIN TRY
        IF @folio IS NULL THROW 50034, 'Error: El folio de la orden no
puede estar vacío.', 1;
        IF @codigo_repuesto IS NULL THROW 50021, 'Error: El código del
repuesto no puede estar vacío.', 1;
        IF @cantidad IS NULL OR @cantidad <= 0 THROW 50037, 'Error: La
cantidad debe ser un número válido mayor a cero.', 1;
        IF NOT EXISTS (SELECT 1 FROM Repuesto WHERE codigo_repuesto =
@codigo_repuesto) THROW 50038, 'Error: El repuesto no existe en el
catálogo.', 1;

        DECLARE @sede CHAR(3);
        SELECT @sede = codigo_sede FROM VDA_Orden_info WHERE
numero_folio = @folio;
        IF @sede IS NULL THROW 50035, 'Error: La orden padre no
existe.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        INSERT INTO VDA_Detalle_Repuesto (numero_folio,
codigo_repuesto, cantidad, codigo_sede)
VALUES (@folio, @codigo_repuesto, @cantidad, @sede);
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

CREATE PROCEDURE sp_ActualizarDetalleRepuesto
    @folio INT, @codigo_repuesto INT, @nueva_cantidad INT
AS BEGIN
SET NOCOUNT ON;
    BEGIN TRY
```




ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
        IF @folio IS NULL THROW 50034, 'Error: El folio de la orden no
puede estar vacío.', 1;
        IF @codigo_repuesto IS NULL THROW 50021, 'Error: El código del
repuesto no puede estar vacío.', 1;
        IF @nueva_cantidad IS NULL OR @nueva_cantidad <= 0 THROW
50037, 'Error: La cantidad debe ser un número válido mayor a cero.',
1;

        IF NOT EXISTS (SELECT 1 FROM VDA_Detalle_Repuesto WHERE
numero_folio = @folio AND codigo_repuesto = @codigo_repuesto)
            THROW 50039, 'Error: El detalle de repuesto no existe en
esta orden.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        UPDATE VDA_Detalle_Repuesto
        SET cantidad = @nueva_cantidad
        WHERE numero_folio = @folio AND codigo_repuesto =
@codigo_repuesto;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

CREATE PROCEDURE sp_EliminarDetalleRepuesto
    @folio INT, @codigo_repuesto INT
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF @folio IS NULL THROW 50034, 'Error: El folio no puede estar
vacío.', 1;
        IF @codigo_repuesto IS NULL THROW 50021, 'Error: El código del
repuesto no puede estar vacío.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        DELETE FROM VDA_Detalle_Repuesto
        WHERE numero_folio = @folio AND codigo_repuesto =
@codigo_repuesto;
        COMMIT TRANSACTION;
    END TRY
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH
END;
GO

/*-----*/
-- 1. LECTURA PARA DASHBOARD
/*-----*/
GO

CREATE PROCEDURE sp_ObtenerDashboardOrdenes
    @sede_filtro CHAR(3),
    @estado_filtro VARCHAR(20) = NULL,
    @fecha_busqueda DATE = NULL
AS BEGIN
SET NOCOUNT ON;
SELECT
    o.numero_folio,
    o.fecha_ingreso,
    o.descripcion_fallo,
    o.estado_orden,
    t.nombre_tecnico + ' ' + t.apellido_tecnico AS nombre_tecnico,
    c.nombre_completo AS nombre_cliente,
    o.codigo_sede,
    l.nivel_encryption,
    l.protocolo_seguridad,
    l.horas_laboratorio
FROM VDA_Orden_info o
LEFT JOIN VDA_Tecnico t ON o.codigo_tecnico = t.codigo_empleado
LEFT JOIN Cliente c ON o.cedula_cliente = c.cedula_cliente
-- S02 consulta a su propia tabla de forma local sin IP
LEFT JOIN TechFixS02.dbo.Orden_labo l ON o.numero_folio =
l.numero_folio
WHERE
    o.codigo_sede = @sede_filtro AND
    (@estado_filtro IS NULL OR @estado_filtro = '' OR
o.estado_orden = @estado_filtro) AND
    (@fecha_busqueda IS NULL OR o.fecha_ingreso = @fecha_busqueda)
ORDER BY o.fecha_ingreso DESC, o.numero_folio DESC;
END;
GO
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
/*-----*/
-- 2. GESTIÓN DE CLIENTES
/*-----*/
CREATE PROCEDURE sp_InsertarCliente
    @cedula VARCHAR(20), @nombre VARCHAR(100), @telefono VARCHAR(20),
    @correo VARCHAR(100)
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF LTRIM(RTRIM(ISNULL(@cedula, ''))) = '' THROW 50001, 'Error:
La cédula no puede estar vacía.', 1;
        IF LTRIM(RTRIM(ISNULL(@nombre, ''))) = '' THROW 50002, 'Error:
El nombre no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@telefono, ''))) = '' THROW 50003,
'Error: El teléfono no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@correo, ''))) = '' THROW 50004, 'Error:
El correo no puede estar vacío.', 1;

        IF LEN(@cedula) < 10 THROW 50005, 'Error: La cédula debe tener
al menos 10 caracteres.', 1;
        IF @correo NOT LIKE '%@__%.__%' THROW 50006, 'Error: Formato
de correo electrónico inválido.', 1;
        IF EXISTS (SELECT 1 FROM Cliente WHERE cedula_cliente =
@cedula) THROW 50007, 'Error: El cliente ya existe en el sistema.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        INSERT INTO Cliente (cedula_cliente, nombre_completo,
telefono_cliente, correo_cliente)
VALUES (@cedula, @nombre, @telefono, @correo);
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

CREATE PROCEDURE sp_ActualizarCliente
    @cedula VARCHAR(20), @nombre VARCHAR(100), @telefono VARCHAR(20),
    @correo VARCHAR(100)
AS BEGIN
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
SET NOCOUNT ON;
BEGIN TRY
    IF LTRIM(RTRIM(ISNULL(@cedula, ''))) = '' THROW 50001, 'Error:
La cédula no puede estar vacía.', 1;
    IF LTRIM(RTRIM(ISNULL(@nombre, ''))) = '' THROW 50002, 'Error:
El nombre no puede estar vacío.', 1;
    IF LTRIM(RTRIM(ISNULL(@telefono, ''))) = '' THROW 50003,
'Error: El teléfono no puede estar vacío.', 1;
    IF LTRIM(RTRIM(ISNULL(@correo, ''))) = '' THROW 50004, 'Error:
El correo no puede estar vacío.', 1;

    IF NOT EXISTS (SELECT 1 FROM Cliente WHERE cedula_cliente =
@cedula) THROW 50008, 'Error: El cliente no existe.', 1;
    IF @correo NOT LIKE '%_@_%._%' THROW 50006, 'Error: Formato
de correo electrónico inválido.', 1;

    BEGIN DISTRIBUTED TRANSACTION;
    UPDATE Cliente
    SET nombre_completo = @nombre, telefono_cliente = @telefono,
correo_cliente = @correo
    WHERE cedula_cliente = @cedula;
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH
END;
GO

CREATE PROCEDURE sp_EliminarCliente
    @cedula VARCHAR(20)
AS BEGIN
SET NOCOUNT ON;
    BEGIN TRY
        IF LTRIM(RTRIM(ISNULL(@cedula, ''))) = '' THROW 50001, 'Error:
La cédula no puede estar vacía.', 1;
        IF EXISTS (SELECT 1 FROM VDA_Orden_info WHERE cedula_cliente =
@cedula)
            THROW 50009, 'Error: No se puede eliminar el cliente
porque tiene órdenes de servicio asociadas.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
        DELETE FROM Cliente WHERE cedula_cliente = @cedula;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

/*-----*/
-- 3. GESTIÓN DE TÉCNICOS
/*-----*/
CREATE PROCEDURE sp_InsertarTecnico
    @cedula VARCHAR(20), @nombre VARCHAR(50), @apellido VARCHAR(50),
    @especialidad VARCHAR(50), @sede CHAR(3)
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF LTRIM(RTRIM(ISNULL(@cedula, ''))) = '' THROW 50010, 'Error:
La cédula del técnico no puede estar vacía.', 1;
        IF LTRIM(RTRIM(ISNULL(@nombre, ''))) = '' THROW 50011, 'Error:
El nombre no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@apellido, ''))) = '' THROW 50012,
'Error: El apellido no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@especialidad, ''))) = '' THROW 50013,
'Error: La especialidad no puede estar vacía.', 1;
        IF LTRIM(RTRIM(ISNULL(@sede, ''))) = '' THROW 50014, 'Error:
La sede no puede estar vacía.', 1;

        IF @sede NOT IN ('S01', 'S02') THROW 50015, 'Error: La sede
debe ser S01 o S02.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        DECLARE @nuevo_codigo INT;
        SELECT @nuevo_codigo = ISNULL(MAX(codigo_empleado), 0) + 1
FROM VDA_Tecnico;

        INSERT INTO VDA_Tecnico (codigo_empleado, cedula_tecnico,
nombre_tecnico, apellido_tecnico, especialidad_tecnico, codigo_sede)
VALUES (@nuevo_codigo, @cedula, @nombre, @apellido,
@especialidad, @sede);
        COMMIT TRANSACTION;
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH
END;
GO

CREATE PROCEDURE sp_ActualizarTecnico
    @codigo INT, @cedula VARCHAR(20), @nombre VARCHAR(50), @apellido
    VARCHAR(50), @especialidad VARCHAR(50)
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF @codigo IS NULL THROW 50016, 'Error: El código de empleado
no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@cedula, ''))) = '' THROW 50010, 'Error:
La cédula del técnico no puede estar vacía.', 1;
        IF LTRIM(RTRIM(ISNULL(@nombre, ''))) = '' THROW 50011, 'Error:
El nombre no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@apellido, ''))) = '' THROW 50012,
'Error: El apellido no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@especialidad, ''))) = '' THROW 50013,
'Error: La especialidad no puede estar vacía.', 1;

        IF NOT EXISTS (SELECT 1 FROM VDA_Tecnico WHERE codigo_empleado
= @codigo) THROW 50017, 'Error: El técnico no existe.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        UPDATE VDA_Tecnico
        SET cedula_tecnico = @cedula, nombre_tecnico = @nombre,
            apellido_tecnico = @apellido, especialidad_tecnico =
@especialidad
        WHERE codigo_empleado = @codigo;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
CREATE PROCEDURE sp_EliminarTecnico
    @codigo INT
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF @codigo IS NULL THROW 50016, 'Error: El código de empleado
no puede estar vacío.', 1;
        IF EXISTS (SELECT 1 FROM VDA_Orden_info WHERE codigo_tecnico =
@codigo)
            THROW 50018, 'Error: No se puede eliminar el técnico
porque tiene órdenes asignadas.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        DELETE FROM VDA_Tecnico WHERE codigo_empleado = @codigo;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

/*-----*/
-- 4. GESTIÓN DE REPUESTOS
/*-----*/
CREATE PROCEDURE sp_InsertarRepuesto
    @nombre VARCHAR(100), @costo DECIMAL(10,2)
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF LTRIM(RTRIM(ISNULL(@nombre, ''))) = '' THROW 50019, 'Error:
El nombre de la pieza no puede estar vacío.', 1;
        IF @costo IS NULL OR @costo < 0 THROW 50020, 'Error: El costo
unitario debe ser válido y no negativo.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        DECLARE @nuevo_codigo INT;
        SELECT @nuevo_codigo = ISNULL(MAX(codigo_repuesto), 0) + 1
FROM Repuesto;

        INSERT INTO Repuesto (codigo_repuesto, nombre_pieza,
costo_unitario)
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
VALUES (@nuevo_codigo, @nombre, @costo);
COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH
END;
GO

CREATE PROCEDURE sp_ActualizarRepuesto
    @codigo INT, @nombre VARCHAR(100), @costo DECIMAL(10,2)
AS BEGIN
SET NOCOUNT ON;
    BEGIN TRY
        IF @codigo IS NULL THROW 50021, 'Error: El código de repuesto
no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@nombre, ''))) = '' THROW 50019, 'Error:
El nombre de la pieza no puede estar vacío.', 1;
        IF @costo IS NULL OR @costo < 0 THROW 50020, 'Error: El costo
unitario debe ser válido y no negativo.', 1;
        IF NOT EXISTS (SELECT 1 FROM Repuesto WHERE codigo_repuesto =
@codigo) THROW 50022, 'Error: El repuesto no existe.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        UPDATE Repuesto
        SET nombre_pieza = @nombre, costo_unitario = @costo
        WHERE codigo_repuesto = @codigo;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

CREATE PROCEDURE sp_EliminarRepuesto
    @codigo INT
AS BEGIN
SET NOCOUNT ON;
    BEGIN TRY
```




ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
        IF @codigo IS NULL THROW 50021, 'Error: El código de repuesto
no puede estar vacío.', 1;
        IF EXISTS (SELECT 1 FROM VDA_Detalle_Repuesto WHERE
codigo_repuesto = @codigo)
            THROW 50023, 'Error: No se puede eliminar el repuesto
porque está asignado a una orden.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        DELETE FROM Repuesto WHERE codigo_repuesto = @codigo;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

/*-----*/
-- 5. GESTIÓN DE ÓRDENES - NODO S02
/*-----*/
CREATE PROCEDURE sp_InsertarOrden
    @fecha DATE, @fallo VARCHAR(255), @estado VARCHAR(20), @tecnico
INT, @cliente VARCHAR(20), @sede CHAR(3),
    @encryptacion VARCHAR(50) = NULL, @protocolo VARCHAR(50) = NULL,
@horas INT = NULL
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF LTRIM(RTRIM(ISNULL(@fallo, ''))) = '' THROW 50024, 'Error:
La descripción del fallo no puede estar vacía.', 1;
        IF LTRIM(RTRIM(ISNULL(@estado, ''))) = '' THROW 50025, 'Error:
El estado de la orden no puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@cliente, ''))) = '' THROW 50026,
'Error: La cédula del cliente no puede estar vacía.', 1;
        IF LTRIM(RTRIM(ISNULL(@sede, ''))) = '' THROW 50027, 'Error:
La sede no puede estar vacía.', 1;
        IF @tecnico IS NULL THROW 50028, 'Error: Debe asignar un
técnico válido.', 1;
        IF @fecha IS NULL SET @fecha = GETDATE();

        IF @estado NOT IN ('Pendiente', 'En Proceso', 'Finalizada')
THROW 50029, 'Error: Estado de orden inválido.', 1;
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
IF @fecha > GETDATE() THROW 50030, 'Error: La fecha de ingreso
no puede ser futura.', 1;
IF NOT EXISTS (SELECT 1 FROM Cliente WHERE cedula_cliente =
@cliente) THROW 50031, 'Error: El cliente no existe.', 1;

DECLARE @sede_tecnico CHAR(3);
SELECT @sede_tecnico = codigo_sede FROM VDA_Tecnico WHERE
codigo_empleado = @tecnico;
IF @sede_tecnico IS NULL THROW 50032, 'Error: El técnico no
existe.', 1;
IF @sede_tecnico <> @sede THROW 50040, 'Error: El técnico
asignado no pertenece a la sede de la orden.', 1;

BEGIN DISTRIBUTED TRANSACTION;

DECLARE @nuevo_folio INT;
IF @sede = 'S01' BEGIN
    SELECT @nuevo_folio = ISNULL(MAX(numero_folio), 99) FROM
VDA_Orden_info WHERE numero_folio % 2 <> 0 AND numero_folio >= 101;
    SET @nuevo_folio = @nuevo_folio + 2;

    INSERT INTO VDA_Orden_info (numero_folio, fecha_ingreso,
descripcion_fallo, estado_orden, codigo_tecnico, cedula_cliente,
codigo_sede)
    VALUES (@nuevo_folio, @fecha, @fallo, @estado, @tecnico,
@cliente, @sede);
END
ELSE IF @sede = 'S02' BEGIN
    SELECT @nuevo_folio = ISNULL(MAX(numero_folio), 98) FROM
VDA_Orden_info WHERE numero_folio % 2 = 0 AND numero_folio >= 100;
    SET @nuevo_folio = @nuevo_folio + 2;

    -- ORDEN DE INSERCIÓN CORREGIDO (Padre primero, hijo
después)
    INSERT INTO TechFixS02.dbo.Orden_labo (numero_folio,
nivel_encryptacion, protocolo_seguridad, horas_laboratorio)
    VALUES (@nuevo_folio, @encryptacion, @protocolo, @horas);

    INSERT INTO VDA_Orden_info (numero_folio, fecha_ingreso,
descripcion_fallo, estado_orden, codigo_tecnico, cedula_cliente,
codigo_sede)
    VALUES (@nuevo_folio, @fecha, @fallo, @estado, @tecnico,
@cliente, @sede);
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
END
ELSE BEGIN
    THROW 50033, 'Error: Sede no reconocida.', 1;
END
SELECT @nuevo_folio AS folio_generado;
COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH
END;
GO

CREATE PROCEDURE sp_ActualizarOrden
    @folio INT, @fallo VARCHAR(255), @estado VARCHAR(20), @tecnico
    INT, @encryptacion VARCHAR(50) = NULL, @protocolo VARCHAR(50) = NULL,
    @horas INT = NULL
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF @folio IS NULL THROW 50034, 'Error: El folio de la orden no
        puede estar vacío.', 1;
        IF LTRIM(RTRIM(ISNULL(@fallo, ''))) = '' THROW 50024, 'Error:
        La descripción del fallo no puede estar vacía.', 1;
        IF LTRIM(RTRIM(ISNULL(@estado, ''))) = '' THROW 50025, 'Error:
        El estado de la orden no puede estar vacío.', 1;
        IF @tecnico IS NULL THROW 50028, 'Error: Debe asignar un
        técnico válido.', 1;
        IF @estado NOT IN ('Pendiente', 'En Proceso', 'Finalizada')
        THROW 50029, 'Error: Estado de orden inválido.', 1;

        DECLARE @sede CHAR(3);
        SELECT @sede = codigo_sede FROM VDA_Orden_info WHERE
        numero_folio = @folio;
        IF @sede IS NULL THROW 50035, 'Error: La orden no existe.', 1;

        DECLARE @sede_tecnico CHAR(3);
        SELECT @sede_tecnico = codigo_sede FROM VDA_Tecnico WHERE
        codigo_empleado = @tecnico;
        IF @sede_tecnico IS NULL THROW 50032, 'Error: El técnico no
        existe.', 1;
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
IF @sede_tecnico <> @sede THROW 50040, 'Error: El técnico
asignado no pertenece a la sede de la orden.', 1;

BEGIN DISTRIBUTED TRANSACTION;
-- En el update el orden no rompe llaves porque el registro ya
existe, pero se mantiene la lógica a la vista.
UPDATE VDA_Orden_info
SET descripcion_fallo = @fallo, estado_orden = @estado,
codigo_tecnico = @tecnico
WHERE numero_folio = @folio;

IF @sede = 'S02' BEGIN
    UPDATE TechFixS02.dbo.Orden_labo
    SET nivel_encryption = @encryption,
    protocolo_seguridad = @protocolo, horas_laboratorio = @horas
    WHERE numero_folio = @folio;
END
COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH
END;
GO

CREATE PROCEDURE sp_EliminarOrden
    @folio INT
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF @folio IS NULL THROW 50034, 'Error: El folio de la orden no
puede estar vacío.', 1;

        DECLARE @sede CHAR(3);
        DECLARE @estadoActual VARCHAR(20);

        SELECT @sede = codigo_sede, @estadoActual = estado_orden
        FROM VDA_Orden_info WHERE numero_folio = @folio;

        IF @sede IS NULL THROW 50035, 'Error: La orden no existe.', 1;
        IF @estadoActual <> 'Pendiente'
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
        THROW 50036, 'Error: Acción denegada. Solo se pueden
eliminar órdenes que se encuentren en estado "Pendiente."', 1;

        BEGIN DISTRIBUTED TRANSACTION;

        -- ORDEN DE ELIMINACIÓN CORREGIDO (Hijos primero, padres al
final)
        -- 1. Se borran los repuestos (Hijo de Orden_info)
        DELETE FROM VDA_Detalle_Repuesto WHERE numero_folio = @folio;

        -- 2. Se borra la Orden general (Hijo de Orden_labo)
        DELETE FROM VDA_Orden_info WHERE numero_folio = @folio;

        -- 3. Se borra el laboratorio (Padre)
        IF @sede = 'S02' BEGIN
            DELETE FROM TechFixS02.dbo.Orden_labo WHERE numero_folio =
@folio;
        END

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
/*-----*/
-- 6. GESTIÓN DE DETALLES DE REPUESTO
/*-----*/
CREATE PROCEDURE sp_InsertarDetalleRepuesto
    @folio INT, @codigo_repuesto INT, @cantidad INT
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF @folio IS NULL THROW 50034, 'Error: El folio de la orden no
puede estar vacío.', 1;
        IF @codigo_repuesto IS NULL THROW 50021, 'Error: El código del
repuesto no puede estar vacío.', 1;
        IF @cantidad IS NULL OR @cantidad <= 0 THROW 50037, 'Error: La
cantidad debe ser un número válido mayor a cero.', 1;
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
        IF NOT EXISTS (SELECT 1 FROM Repuesto WHERE codigo_repuesto =
@codigo_repuesto) THROW 50038, 'Error: El repuesto no existe en el
catálogo.', 1;

        DECLARE @sede CHAR(3);
        SELECT @sede = codigo_sede FROM VDA_Orden_info WHERE
numero_folio = @folio;
        IF @sede IS NULL THROW 50035, 'Error: La orden padre no
existe.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        INSERT INTO VDA_Detalle_Repuesto (numero_folio,
codigo_repuesto, cantidad, codigo_sede)
VALUES (@folio, @codigo_repuesto, @cantidad, @sede);
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

CREATE PROCEDURE sp_ActualizarDetalleRepuesto
    @folio INT, @codigo_repuesto INT, @nueva_cantidad INT
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF @folio IS NULL THROW 50034, 'Error: El folio de la orden no
puede estar vacío.', 1;
        IF @codigo_repuesto IS NULL THROW 50021, 'Error: El código del
repuesto no puede estar vacío.', 1;
        IF @nueva_cantidad IS NULL OR @nueva_cantidad <= 0 THROW
50037, 'Error: La cantidad debe ser un número válido mayor a cero.',
1;

        IF NOT EXISTS (SELECT 1 FROM VDA_Detalle_Repuesto WHERE
numero_folio = @folio AND codigo_repuesto = @codigo_repuesto)
            THROW 50039, 'Error: El detalle de repuesto no existe en
esta orden.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        UPDATE VDA_Detalle_Repuesto
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
        SET cantidad = @nueva_cantidad
        WHERE numero_folio = @folio AND codigo_repuesto =
@codigo_repuesto;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO

CREATE PROCEDURE sp_EliminarDetalleRepuesto
    @folio INT, @codigo_repuesto INT
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF @folio IS NULL THROW 50034, 'Error: El folio no puede estar
vacío.', 1;
        IF @codigo_repuesto IS NULL THROW 50021, 'Error: El código del
repuesto no puede estar vacío.', 1;

        BEGIN DISTRIBUTED TRANSACTION;
        DELETE FROM VDA_Detalle_Repuesto
        WHERE numero_folio = @folio AND codigo_repuesto =
@codigo_repuesto;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
```